

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN 2
KHAI THÁC TẬP PHỔ BIẾN

Môn học: Khai thác dữ liệu và ứng dụng

Sinh viên thực hiện:

Trần Thanh Đạt (23120030)

Phạm Ngọc Duy (23120035)

Lê Minh Hải (23120041)

Huỳnh Đặng Ngọc Hân (23120042)

Nguyễn Mạnh Thắng (23120084)

Giáo viên hướng dẫn:

GS. TS. Lê Hoài Bắc

ThS. Nguyễn Thị Thu Hằng

ThS. Nguyễn Ngọc Đức

ThS. Lê Nhật Nam

TP. Hồ Chí Minh - Ngày 14 tháng 6 năm 2026

Mục lục

DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT	iii
DANH MỤC CÁC THUẬT NGỮ	v
DANH MỤC CÁC HÌNH VẼ, ĐỒ THỊ	vii
1 Chương 1: Nền tảng lý thuyết	1
1.1 Bài toán Frequent Itemset Mining	1
1.2 Phân tích thuật toán được chọn: GenMax	3
1.2.1 Ý tưởng cốt lõi	3
1.2.2 Cấu trúc dữ liệu chính	3
1.2.3 Giả mã	4
1.2.4 Phân tích độ phức tạp	6
1.2.5 So sánh lịch sử	6
2 Chương 2: Ví dụ minh họa tay thuật toán GenMax	7
2.1 Cơ sở dữ liệu đồ chơi và ngưỡng hỗ trợ	7
2.2 Bước 1: Chuyển sang biểu diễn dọc bằng tidset	8
2.3 Bước 2: Duyệt cây tìm kiếm của GenMax	8
2.3.1 Nhánh có tiền tố A	9
2.3.2 Nhánh có tiền tố B	9
2.3.3 Nhánh có tiền tố C	10
2.3.4 Nhánh có tiền tố D	10
2.3.5 Nhánh có tiền tố E và bước cắt tỉa cuối	11
2.4 Kết quả tập phổ biến tối đại	11
2.5 Kiểm tra chéo bằng cách liệt kê thủ công tập phổ biến	11
2.6 Tình huống đặc biệt: CSDL chỉ có một nhánh	12
3 Chương 3: Thiết kế và Cài đặt Lỗi Thuật toán GenMax	13
3.1 Nhiệm vụ và Mục tiêu	13
3.2 Cấu trúc dữ liệu dọc và Tối ưu hóa Bộ nhớ/Tốc độ	14

3.3	Thiết kế giải thuật GenMax và Cơ chế Cắt tỉa	14
3.3.1	Khởi tạo và Sắp xếp ứng viên ban đầu	14
3.3.2	Kỹ thuật Diffset (<code>fi_diffset_combine</code>)	15
3.3.3	Chiến lược Progressive Focusing và đệ quy quay lui (<code>lmfi_backtrack</code>)	15
3.4	Kiểm thử và Đánh giá Correctness	16
4	Chương 4: Thực nghiệm và Đánh giá	16
4.1	Tập dữ liệu benchmark	16
4.2	Thực nghiệm và kết quả	17
4.2.1	Thí nghiệm 1: Tính đúng đắn (Correctness)	17
4.2.2	Thí nghiệm 2: Thời gian chạy theo Minsup	18
4.2.3	Thí nghiệm 3: Số lượng tập phổ biến tối đại theo Minsup	20
4.2.4	Thí nghiệm 4: Sử dụng bộ nhớ	22
4.2.5	Thí nghiệm 5: Đánh giá khả năng mở rộng	23
4.2.6	Thí nghiệm 6: Ảnh hưởng của độ dài giao dịch trung bình	24
4.3	Phân tích kết quả và đề xuất	25
4.3.1	Giải thích lý thuyết xu hướng thời gian chạy và bộ nhớ	25
4.3.2	Phân tích so sánh thuật toán GenMax	26
4.3.3	Đề xuất hướng tối ưu hóa	27
5	Chương 5: Ứng dụng	27
5.1	Bài toán phân tích giỏ hàng	27
5.2	Kết quả tập phổ biến tối đại	28
5.3	Top-10 luật kết hợp theo lift	28
5.4	Ý nghĩa kinh doanh của các luật	29
5.5	Nhận xét tổng hợp	30
	Tài liệu	31

DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT

$\mathcal{I}$	Tập hợp tất cả các mục dữ liệu riêng biệt trong hệ thống
$\mathcal{D}$	Cơ sở dữ liệu giao dịch, bao gồm tập hợp các bản ghi giao dịch
T_j, S_i	Các bản ghi giao dịch trong cơ sở dữ liệu, bản thân nó là một tập con của $\mathcal{I}$
X, Y	Ký hiệu đại diện cho các tập mục (itemset) bất kỳ
k	Số lượng phần tử có trong một tập mục
$ X $	Lực lượng của tập hợp X , chỉ định số phần tử của X
$sup(X)$	Độ hỗ trợ tuyệt đối của tập mục X
$minsup$	Ngưỡng độ hỗ trợ tối thiểu do người dùng quy định
$minconf$	Ngưỡng độ tin cậy tối thiểu cho luật kết hợp
$\mathcal{FI}$	Tập hợp tất cả các tập phổ biến
$\mathcal{CFI}$	Tập hợp tất cả các tập phổ biến đóng
$\mathcal{MFI}$	Tập hợp tất cả các tập phổ biến tối đại
F_1	Tập hợp các mục phổ biến có kích thước bằng 1 (1-itemset phổ biến)
$IF(x)$	Tần suất không tương thích (Incompatibility Frequency) của mục x
l	Độ sâu hiện tại của cây tìm kiếm đệ quy
I_l	Tập mục hiện tại đang được xét tại độ sâu l
$LMFI_l$	Tập hợp các tập phổ biến tối đại cục bộ tại độ sâu l
$t(X)$	Tập mã định danh giao dịch (TIDset) chứa tập mục X
$d(X)$	Tập biệt số (Diffset) của tập mục X
C_l	Tập hợp các phần tử ứng viên tại độ sâu l trong quá trình tìm kiếm quay lui
P_l	Tập hợp các phần tử có khả năng nối tiếp để mở rộng tập mục tại độ sâu l
CPU	Central Processing Unit - Bộ vi xử lý trung tâm

FIM	Frequent Itemset Mining - Khai thác tập phổ biến
JIT	Just-In-Time - Kỹ thuật biên dịch mã nguồn tại thời điểm thực thi
JVM	Java Virtual Machine - Máy ảo Java
RAM	Random Access Memory - Bộ nhớ truy cập ngẫu nhiên
SPMF	Sequential Pattern Mining Framework - Thư viện khai phá mẫu tuần tự

DANH MỤC CÁC THUẬT NGỮ

A/B Testing	Kiểm thử A/B (Phương pháp thử nghiệm phân tách)
Absolute Support	Độ hỗ trợ tuyệt đối
Apriori Property	Tính chất Apriori
Association Rules	Luật kết hợp
Backtracking Search	Tìm kiếm quay lui
Benchmark	Tập dữ liệu hoặc hệ thống dùng làm tiêu chuẩn đánh giá
BitSet / BitArray	Cấu trúc dữ liệu mảng bit
Breadth-First Search (BFS)	Tìm kiếm theo chiều rộng
Candidate Generation	Sinh ứng viên
Closed Itemset	Tập đóng
Confidence	Độ tin cậy
Correctness	Tính đúng đắn (Độ chính xác của kết quả đầu ra)
Dense Database	Cơ sở dữ liệu dày đặc
Depth-First Search (DFS)	Tìm kiếm theo chiều sâu
Diffset	Tập biệt số
Downward Closure Property	Tính chất đơn điệu giảm
Frequent Itemset	Tập phổ biến
Frequent Itemset Mining (FIM)	Khai thác tập phổ biến
Incompatibility Frequency	Tần suất không tương thích
Intersection	Phép giao cắt
Item	Mục dữ liệu
Itemset	Tập mục
IT-tree (Itemset-Tidset tree)	Cây không gian tìm kiếm tập mục kết hợp mã giao dịch

Lift	Độ nâng (Chỉ số đo lường sức mạnh của luật kết hợp)
Lock Contention	Tranh chấp khóa / Tranh chấp bộ nhớ giữa các luồng
Market Basket Analysis	Phân tích giỏ hàng
Maximal Itemset	Tập tối đại
Minimum Support (minsup)	Ngưỡng hỗ trợ tối thiểu
Multi-threading	Đa luồng (Kỹ thuật xử lý song song)
Peak Memory	Bộ nhớ tiêu thụ đỉnh
Progressive Focusing	Tập trung lũy tiến
Pruning	Tỉa nhánh
Relative Support	Độ hỗ trợ tương đối
Roaring Bitmaps	Cấu trúc dữ liệu nén mảng bit hiệu năng cao
Scalability	Khả năng mở rộng / Khả năng co giãn của thuật toán
Search Space	Không gian tìm kiếm
Single Path	Nhánh đơn duy nhất
Sparse Database	Cơ sở dữ liệu thưa
Support	Độ hỗ trợ
Transaction Database	Cơ sở dữ liệu giao dịch
Transaction ID (TID)	Mã định danh giao dịch
TIDset	Tập mã định danh giao dịch
Vertical Database Format	Định dạng dữ liệu dọc

Danh sách hình vẽ

1	Minh họa cây tìm kiếm và so sánh cấu trúc dữ liệu Tidset và Diffset (mở rộng từ nhánh A)	4
2	Thời gian chạy trên Chess	19
3	Thời gian chạy trên Mushroom	19
4	Thời gian chạy trên Retail	19
5	Thời gian chạy trên Accidents	19
6	Số MFI sinh ra trên Chess	21
7	Số MFI sinh ra trên Mushroom	21
8	Số MFI sinh ra trên Retail	21
9	Số MFI sinh ra trên Accidents	21
10	Khả năng mở rộng (Thời gian)	23
11	Khả năng mở rộng (Bộ nhớ)	23
12	Ảnh hưởng độ dài giao dịch (Thời gian)	24
13	Ảnh hưởng độ dài giao dịch (Bộ nhớ)	24

1 Chương 1: Nền tảng lý thuyết

1.1 Bài toán Frequent Itemset Mining

Cho $\mathcal{I} = \{i_1, i_2, \dots, i_m\}$ là tập hợp gồm m mục hàng riêng biệt, trong đó mỗi mục hàng được gọi là một mục. Một tập mục $X \subseteq \mathcal{I}$ gồm k phần tử ($k \leq m$) được gọi là một k -tập mục.

Dưới đây là các khái niệm và định nghĩa hình thức nền tảng của bài toán khai thác tập phổ biến.

Định nghĩa 1.1 (Cơ sở dữ liệu giao dịch). Cơ sở dữ liệu giao dịch $\mathcal{D} = \{T_1, T_2, \dots, T_n\}$ là tập hợp gồm n bản ghi phân biệt. Mỗi bản ghi $T_j \in \mathcal{D}$ ($1 \leq j \leq n$) được gọi là một giao dịch. Mỗi giao dịch T_j là một tập con của $\mathcal{I}$ ($T_j \subseteq \mathcal{I}$) và được gắn liền với một mã định danh giao dịch duy nhất.

Định nghĩa 1.2 (Độ hỗ trợ). Cho một tập mục $X \subseteq \mathcal{I}$. Độ hỗ trợ tuyệt đối của X , ký hiệu là $sup(X)$, là số lượng giao dịch trong cơ sở dữ liệu $\mathcal{D}$ có chứa X :

$$sup(X) = |\{T_j \in \mathcal{D} \mid X \subseteq T_j\}| \quad (1)$$

Bên cạnh đó, độ hỗ trợ tương đối của X được tính bằng tỷ lệ phần trăm số giao dịch chứa X trên tổng số giao dịch của $\mathcal{D}$:

$$rel_sup(X) = \frac{sup(X)}{|\mathcal{D}|} \quad (2)$$

Định nghĩa 1.3 (Tập phổ biến). Một tập mục $X \subseteq \mathcal{I}$ được gọi là tập phổ biến nếu độ hỗ trợ của nó lớn hơn hoặc bằng một ngưỡng hỗ trợ tối thiểu ($minsup$) do người dùng quy định trước:

$$sup(X) \geq minsup \quad (3)$$

Tập hợp tất cả các tập mục phổ biến trong cơ sở dữ liệu $\mathcal{D}$ được ký hiệu là $\mathcal{FI}$.

Định nghĩa 1.4 (Tập đóng và Tập tối đại). Dựa trên khái niệm tập phổ biến, không gian kết quả được thu gọn và phân loại chặt chẽ hơn thành hai tập hợp:

- **Tập đóng:** Một tập mục X được gọi là tập phổ biến đóng nếu $X \in \mathcal{FI}$ và không tồn tại bất kỳ tập cha $Y \supset X$ nào có cùng độ hỗ trợ với nó ($\forall Y \supset X : sup(Y) < sup(X)$). Tập hợp này được ký hiệu là $\mathcal{CFI}$.

- **Tập tối đại:** Một tập mục X được gọi là tập phổ biến tối đại nếu $X \in \mathcal{FI}$ và không tồn tại bất kỳ tập cha $Y \supset X$ nào cũng là tập phổ biến ($\forall Y \supset X : \sup(Y) < \text{minsup}$). Tập hợp này được ký hiệu là $\mathcal{MFI}$.

Mối quan hệ bao hàm giữa ba loại tập mục được thể hiện hình thức qua biểu thức toán học:

$$\mathcal{FI} \supseteq \mathcal{CFI} \supseteq \mathcal{MFI} \quad (4)$$

Chứng minh quan hệ bao hàm:

Chứng minh. Ta chứng minh quan hệ này qua hai bước:

1. **Chứng minh $\mathcal{FI} \supseteq \mathcal{CFI}$:** Theo định nghĩa, một tập mục $X \in \mathcal{CFI}$ trước hết phải là một tập phổ biến ($X \in \mathcal{FI}$). Do đó, hiển nhiên ta có $\mathcal{CFI} \subseteq \mathcal{FI}$.
2. **Chứng minh $\mathcal{CFI} \supseteq \mathcal{MFI}$:** Giả sử $X \in \mathcal{MFI}$, tức X là một tập phổ biến tối đại. Theo định nghĩa:

$$X \in \mathcal{FI} \quad \text{và} \quad \forall Y \supset X \implies \sup(Y) < \text{minsup}$$

Vì $X \in \mathcal{FI}$, ta có $\sup(X) \geq \text{minsup}$. Kết hợp hai điều kiện trên, với mọi tập cha $Y \supset X$, ta luôn có:

$$\sup(Y) < \text{minsup} \leq \sup(X) \implies \sup(Y) < \sup(X)$$

Điều này đáp ứng chính xác định nghĩa của tập phổ biến đóng ($\mathcal{CFI}$), nghĩa là không tồn tại tập cha nào có cùng độ hỗ trợ với X . Do đó, $X \in \mathcal{CFI}$, suy ra $\mathcal{MFI} \subseteq \mathcal{CFI}$.

Từ (1) và (2), ta có chuỗi quan hệ bao hàm: $\mathcal{FI} \supseteq \mathcal{CFI} \supseteq \mathcal{MFI}$. □

Tính chất 1.1 (Tính chất Apriori). *Tính chất đơn điệu giảm của độ hỗ trợ phát biểu rằng: Mọi tập con của một tập phổ biến cũng phải là một tập phổ biến. Ngược lại, nếu một tập mục là không phổ biến thì mọi tập cha trực tiếp của nó cũng không thể phổ biến.*

$$\forall X \subseteq Y : \sup(Y) \geq \text{minsup} \implies \sup(X) \geq \text{minsup} \quad (5)$$

Chứng minh. Giả sử có hai tập mục X và Y thỏa mãn điều kiện $X \subseteq Y$. Xét một giao dịch bất kỳ $T_j \in \mathcal{D}$. Nếu giao dịch T_j chứa tập mục Y ($Y \subseteq T_j$), thì do tính chất bao hàm ($X \subseteq Y$), giao dịch T_j chắc chắn cũng phải chứa tập mục X ($X \subseteq T_j$).

Từ logic trên, tập hợp các giao dịch chứa Y luôn là tập con của tập hợp các giao dịch chứa X . Xét về mặt lực lượng của hai tập hợp giao dịch, ta có:

$$|\{T_j \in \mathcal{D} \mid Y \subseteq T_j\}| \leq |\{T_j \in \mathcal{D} \mid X \subseteq T_j\}| \quad (6)$$

Điều này tương đương với bất đẳng thức: $\text{sup}(Y) \leq \text{sup}(X)$. Do đó, nếu hệ thống xác định Y là một tập phổ biến ($\text{sup}(Y) \geq \text{minsup}$), thì hiển nhiên X cũng bắt buộc phải là một tập phổ biến ($\text{sup}(X) \geq \text{minsup}$). Tính chất được chứng minh hoàn toàn. $\square$

1.2 Phân tích thuật toán được chọn: GenMax

1.2.1 Ý tưởng cốt lõi

GenMax [3] là một thuật toán khai phá dữ liệu hiệu quả, sử dụng chiến lược tìm kiếm quay lui để liệt kê chính xác các tập phổ biến tối đại.

Ý tưởng trực quan của thuật toán dựa trên việc cắt tỉa mạnh mẽ không gian tìm kiếm nhánh cận: nếu một tập mục ứng viên cùng với tất cả các phần tử có thể kết hợp với nó đã là tập con của một tập tối đại đã được tìm thấy trước đó, GenMax sẽ ngay lập tức cắt bỏ toàn bộ nhánh tìm kiếm này.

Điểm khác biệt đột phá: Khác với thuật toán Apriori hay FP-Growth vốn phải duyệt qua toàn bộ không gian tập phổ biến khổng lồ, GenMax chỉ nhắm thẳng đến tập tối đại. So với thuật toán Mafia chỉ tìm ra siêu tập của tập tối đại và yêu cầu bước hậu xử lý rất tốn kém, GenMax tích hợp việc kiểm tra tính tối đại ngay trong quá trình khai phá, giúp trả về chính xác tập tối đại mà không cần bước lọc lại. Để làm được điều này với tốc độ cao, GenMax sử dụng một kỹ thuật độc quyền gọi là Tập trung lũy tiến để thu hẹp danh sách các tập tối đại cần kiểm tra ở mỗi bước đệ quy.

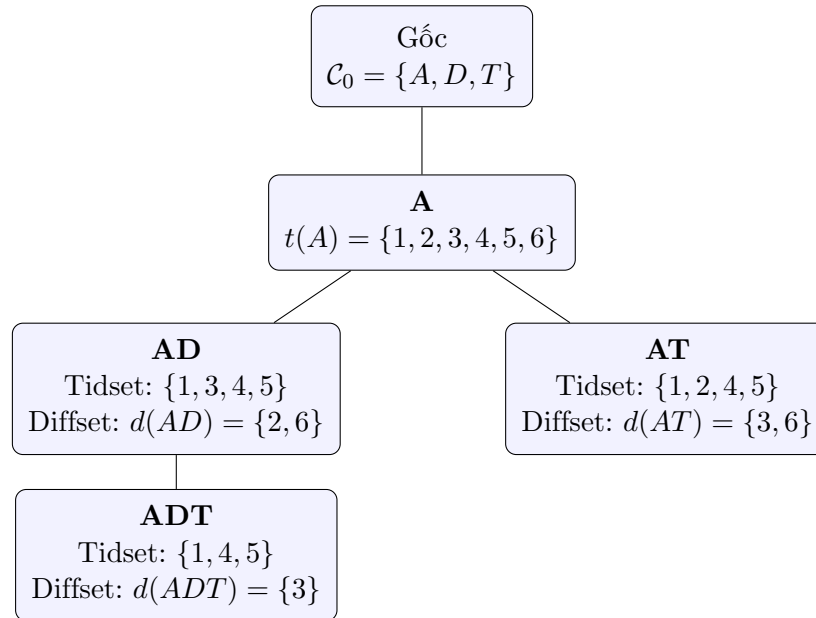
1.2.2 Cấu trúc dữ liệu chính

GenMax sử dụng định dạng dữ liệu dọc, nhưng thay vì lưu trữ toàn bộ mã giao dịch cho mỗi tập mục, nó sử dụng một cấu trúc dữ liệu đặc trưng và tối ưu hơn rất nhiều gọi là Diffset [4].

Diffset chỉ lưu trữ sự khác biệt giữa tập cha và tập con khi mở rộng độ dài của tập mục. Điều này giúp kích thước của bộ nhớ cần sử dụng giảm đi đáng kể và biến các phép giao cắt phức tạp thành các phép trừ tập hợp đơn giản, cực nhanh.

Ví dụ minh họa cấu trúc Diffset: Giả sử ta có tập giao dịch gốc gồm 6 giao dịch, với các tập

giao dịch cụ thể chứa các phần tử được thể hiện trên cây tìm kiếm hình 1.



Hình 1: Minh họa cây tìm kiếm và so sánh cấu trúc dữ liệu Tidset và Diffset (mở rộng từ nhánh A)

Từ sơ đồ trên ta thấy: - Thay vì giao hai Tidset $t(AD)$ và $t(T)$ để tìm $t(ADT)$, thuật toán chỉ cần lan truyền Diffset:

$$d(ADT) = d(AT) \setminus d(AD) = \{3, 6\} \setminus \{2, 6\} = \{3\}$$

- Độ hỗ trợ của ADT được tính cực nhanh bằng phép trừ:

$$sup(ADT) = sup(AD) - |d(ADT)| = 4 - 1 = 3$$

Điều này giúp tiết kiệm bộ nhớ khi đi sâu xuống các nhánh dưới của cây tìm kiếm do lực lượng của Diffset giảm dần đáng kể.

1.2.3 Giải mã

Giải mã dưới đây mô tả luồng thực thi đệ quy cốt lõi của GenMax kết hợp kỹ thuật Tập trung lũy tiến để cắt tỉa nhánh (Thuật toán 1) và hàm phụ trợ tạo danh sách ứng viên thông qua hiệu các Diffset (Thuật toán 2).

Algorithm 1 Luật đệ quy GenMax: LMFI-BACKTRACK($I_l, C_l, LMFI_l, l$)

```

1: Input: Tập hiện tại  $I_l$ , tập các phần tử ứng viên  $C_l$ , danh sách các tập tối đại cục bộ  $LMFI_l$ , độ sâu  $l$ .
2: Output: Cập nhật danh sách tập tối đại.
3: for all  $x \in C_l$  do
4:    $I_{l+1} \leftarrow I_l \cup \{x\}$  ▷ Mở rộng tập mục thêm 1 phần tử
5:    $P_{l+1} \leftarrow \{y \in C_l \mid y > x\}$  ▷ Tạo tập các phần tử có thể nối tiếp
6:   if  $I_{l+1} \cup P_{l+1}$  là tập con của một tập hợp trong  $LMFI_l$  then
7:     return ▷ Cắt tỉa: Toàn bộ nhánh này và các nhánh tiếp theo đã bị bao hàm (HUT)
8:   end if
9:    $LMFI_{l+1} \leftarrow \emptyset$  ▷ Khởi tạo tập tối đại cục bộ cho nhánh con
10:   $C_{l+1} \leftarrow \text{FI-DIFFSET-COMBINE}(I_{l+1}, P_{l+1})$  ▷ Lọc các phần tử phổ biến bằng Diffset
11:  if  $C_{l+1}$  rỗng then ▷ Đã chạm đến nút lá và không thể mở rộng thêm
12:    if  $I_{l+1}$  không phải tập con của bất kỳ tập nào trong  $LMFI_l$  then
13:       $LMFI_l \leftarrow LMFI_l \cup I_{l+1}$  ▷ Lưu nhận  $I_{l+1}$  là một tập tối đại mới
14:    end if
15:  else
16:     $LMFI_{l+1} \leftarrow \{M \in LMFI_l \mid x \in M\}$  ▷ Tập trung lũy tiến: Thu hẹp kiểm tra
17:    LMFI-BACKTRACK( $I_{l+1}, C_{l+1}, LMFI_{l+1}, l + 1$ ) ▷ Đệ quy xuống sâu hơn
18:     $LMFI_l \leftarrow LMFI_l \cup LMFI_{l+1}$  ▷ Gộp kết quả từ nhánh con lên nhánh cha
19:  end if
20: end for

```

Algorithm 2 Hàm kết hợp và tính độ hỗ trợ bằng Diffset: FI-DIFFSET-COMBINE(I_{l+1}, P_{l+1})

```

1: Input: Tập mục hiện tại  $I_{l+1}$  (mới được mở rộng bằng  $x$ ), tập các phần tử ứng viên tiềm năng  $P_{l+1}$ .
2: Output: Tập ứng viên mới  $C_{l+1}$  chứa các phần tử phổ biến cùng Diffset tương ứng.
3:  $C_{l+1} \leftarrow \emptyset$ 
4:  $x \leftarrow$  phần tử cuối cùng của  $I_{l+1}$ 
5: for all  $y \in P_{l+1}$  do
6:   if  $l == 0$  then ▷ Ở mức gốc, sử dụng Tidset để tính Diffset đầu tiên
7:      $d(I_{l+1} \cup \{y\}) \leftarrow t(x) \setminus t(y)$ 
8:   else ▷ Ở các mức sâu hơn, lan truyền Diffset từ các mức trước
9:      $d(I_{l+1} \cup \{y\}) \leftarrow d(y) \setminus d(x)$ 
10:  end if
11:   $sup(I_{l+1} \cup \{y\}) \leftarrow sup(I_{l+1}) - |d(I_{l+1} \cup \{y\})|$ 
12:  if  $sup(I_{l+1} \cup \{y\}) \geq minsup$  then
13:    Thêm  $y$  vào  $C_{l+1}$  với thuộc tính hỗ trợ và Diffset vừa tính.
14:  end if
15: end for
16: Sắp xếp các phần tử trong  $C_{l+1}$  theo thứ tự độ hỗ trợ tăng dần (Heuristic reordering).
17: return  $C_{l+1}$ 

```

1.2.4 Phân tích độ phức tạp

- **Độ phức tạp thời gian:**

- *Trường hợp tốt nhất:* $\mathcal{O}(m \times n)$ khi dữ liệu hoàn toàn không có tập phổ biến nào với m là số phần tử và n là số giao dịch. Thuật toán dừng ngay tại lớp đầu tiên sau khi quét cơ sở dữ liệu.
- *Trường hợp trung bình:* $\mathcal{O}(|\mathcal{MFI}| \times k \times m')$ với $|\mathcal{MFI}|$ là số lượng tập phổ biến tối đại thực tế, k là kích thước trung bình của các tập tối đại này và m' là số lượng phép kiểm tra bao hàm tập con cần thực hiện. Nhờ kỹ thuật Tập trung lũy tiến và tính toán nhanh bằng Diffset, số lượng phép tính giao cắt thực tế giảm đi cực kỳ nhiều so với việc duyệt cạn.
- *Trường hợp xấu nhất:* $\mathcal{O}(2^m)$ khi mọi tổ hợp của m mục đều là tập phổ biến (dữ liệu cực kỳ dày đặc).

- **Độ phức tạp không gian:** GenMax cần không gian bộ nhớ tối ưu. Nhờ sử dụng cấu trúc Diffset thay vì lưu toàn bộ các tập giao dịch gốc, bộ nhớ được tiết kiệm triệt để và hiếm khi vượt quá dung lượng cho phép.
- **Các yếu tố ảnh hưởng hiệu năng:** Hiệu năng của GenMax đặc biệt nhạy cảm với sự phân bố của các tập tối đại. Thuật toán chạy cực nhanh trên dữ liệu có phân bố đối xứng hoặc lệch trái, nhưng chậm hơn trên các dữ liệu có chiều dài giao dịch cực kỳ lớn hoặc cấu trúc chằng chịt.

1.2.5 So sánh lịch sử

Thuật toán GenMax đánh dấu một bước tiến lớn trong dòng thời gian phát triển của các thuật toán khai thác tập phổ biến.

- **Giải quyết hạn chế của thuật toán trước:** Trước GenMax, thuật toán MaxMiner [1] phải duyệt theo chiều rộng trên dữ liệu ngang nên bị nghẽn cổ chai khi đếm độ hỗ trợ. FP-Growth dùng cấu trúc cây nhưng lại khai phá mọi tập phổ biến, gây bùng nổ tổ hợp với dữ liệu dài. Gần nhất là Mafia [2] dùng vector nhị phân rất nhanh, nhưng lại sinh ra siêu tập của tập tối đại, dẫn đến việc phải có một bước hậu xử lý phức tạp để loại bỏ các tập thừa, làm treo

hệ thống khi số lượng lớn. GenMax giải quyết triệt để vấn đề này bằng cách tích hợp cắt tỉa ngay khi duyệt thông qua kỹ thuật Tập trung lũy tiến, cam kết trả về chính xác tập tối đại duy nhất.

- **Hạn chế để lại:** Mặc dù vượt trội, GenMax vẫn gặp khó khăn trên dữ liệu có đặc trưng một tập mục nhỏ lại là tập con của hàng ngàn tập tối đại rất dài. Đặc tính này làm vô hiệu hóa sức mạnh của Tập trung lũy tiến vì danh sách tập tối đại cục bộ cần kiểm tra không thu hẹp được bao nhiêu, buộc GenMax phải dùng đến phần lớn thời gian tính toán chỉ để kiểm tra phép bao hàm. Đây chính là bài toán để ngỏ cho các thuật toán lai sau này.

2 Chương 2: Ví dụ minh họa tay thuật toán GenMax

Chương này trình bày một ví dụ nhỏ để chạy tay thuật toán GenMax. Mục tiêu là theo dõi đầy đủ các bước chính: chuyển cơ sở dữ liệu sang dạng dọc bằng *tidset*, tạo cây tìm kiếm theo tiền tố, giao các *tidset* để tính support, cập nhật tập phổ biến tối đại và kiểm tra chéo kết quả bằng cách liệt kê thủ công các tập phổ biến.

2.1 Cơ sở dữ liệu đồ chơi và ngưỡng hỗ trợ

Xét tập mục $\mathcal{I} = \{A, B, C, D, E, F\}$ và cơ sở dữ liệu giao dịch $\mathcal{D}$ gồm 6 giao dịch như Bảng 3. Chọn ngưỡng hỗ trợ tối thiểu $minsup = 3$, nghĩa là một tập mục được xem là phổ biến nếu xuất hiện trong ít nhất 3 giao dịch.

Bảng 3: Cơ sở dữ liệu giao dịch ban đầu

Mã giao dịch	Các mục xuất hiện trong giao dịch
T_1	$\{A, B, C, D\}$
T_2	$\{A, B, C\}$
T_3	$\{A, B, C, F\}$
T_4	$\{D, E, F\}$
T_5	$\{D, E\}$
T_6	$\{D, E\}$

GenMax khai thác các tập phổ biến tối đại. Thay vì sinh tất cả tập phổ biến rồi mới lọc tối đại, thuật toán duyệt cây theo chiều sâu và dùng các tập tối đại đã tìm được để cắt tỉa những nhánh chắc chắn không thể tạo ra tập tối đại mới.

Trong ví dụ này, để dễ theo dõi khi chạy tay, các mục được xét theo thứ tự từ điển $A \prec B \prec C \prec D \prec E$.

2.2 Bước 1: Chuyển sang biểu diễn dọc bằng tidset

Đầu tiên, với mỗi mục, ta liệt kê tập mã giao dịch chứa mục đó. Bảng 4 là biểu diễn dọc của cơ sở dữ liệu.

Bảng 4: Tidset và support của từng mục đơn

Mục	Tidset	Support	Kết luận
A	$\{1, 2, 3\}$	3	Phổ biến
B	$\{1, 2, 3\}$	3	Phổ biến
C	$\{1, 2, 3\}$	3	Phổ biến
D	$\{1, 4, 5, 6\}$	4	Phổ biến
E	$\{4, 5, 6\}$	3	Phổ biến
F	$\{3, 4\}$	2	Không phổ biến

Sau bước này, chỉ còn lại tập ứng viên ban đầu:

$$tail(\emptyset) = \{A, B, C, D, E\}. \quad (7)$$

Mục F bị loại do $sup(F) = 2 < 3$. Theo tính chất Apriori, mọi tập mục chứa F đều không thể phổ biến.

2.3 Bước 2: Duyệt cây tìm kiếm của GenMax

Mỗi nút trong cây GenMax được biểu diễn bởi một tiền tố P và một danh sách mở rộng $tail(P)$. Khi xét mở rộng $P \cup \{x\}$, thuật toán tính:

$$tidset(P \cup \{x\}) = tidset(P) \cap tidset(x). \quad (8)$$

Nếu support của tập mới nhỏ hơn $minsup$, nhánh đó bị cắt. Nếu không còn mục nào trong $tail$ có thể mở rộng một tập phổ biến, tập hiện tại là ứng viên tối đại và được so sánh với tập MFI hiện có.

Ban đầu:

$$P = \emptyset, \quad tail(P) = \{A, B, C, D, E\}, \quad MFI = \emptyset. \quad (9)$$

2.3.1 Nhánh có tiền tố A

Với $P = \{A\}$, ta có $tidset(A) = \{1, 2, 3\}$ và $tail(A) = \{B, C, D, E\}$. Các phép giao tidset ở mức đầu của nhánh A được trình bày trong Bảng 5.

Bảng 5: Các mở rộng trực tiếp của nhánh $\{A\}$

Tập xét	Tidset sau khi giao	Support	Kết luận
$\{A, B\}$	$\{1, 2, 3\} \cap \{1, 2, 3\} = \{1, 2, 3\}$	3	Phổ biến
$\{A, C\}$	$\{1, 2, 3\} \cap \{1, 2, 3\} = \{1, 2, 3\}$	3	Phổ biến
$\{A, D\}$	$\{1, 2, 3\} \cap \{1, 4, 5, 6\} = \{1\}$	1	Không phổ biến
$\{A, E\}$	$\{1, 2, 3\} \cap \{4, 5, 6\} = \emptyset$	0	Không phổ biến

Ta tiếp tục đi sâu theo nhánh $\{A, B\}$ vì đây là mở rộng phổ biến:

- $tidset(A, B, C) = \{1, 2, 3\} \cap \{1, 2, 3\} = \{1, 2, 3\}$ nên $sup(A, B, C) = 3$. Do đó $\{A, B, C\}$ là một tập phổ biến 3 phần tử.
- $tidset(A, B, D) = \{1, 2, 3\} \cap \{1, 4, 5, 6\} = \{1\}$ nên $sup(A, B, D) = 1 < 3$.
- $tidset(A, B, E) = \{1, 2, 3\} \cap \{4, 5, 6\} = \emptyset$ nên $sup(A, B, E) = 0 < 3$.

Tại nút $\{A, B, C\}$, ta thử các mở rộng còn lại:

$$tidset(A, B, C, D) = \{1, 2, 3\} \cap \{1, 4, 5, 6\} = \{1\}, \quad sup(A, B, C, D) = 1 < 3, \quad (10)$$

$$tidset(A, B, C, E) = \{1, 2, 3\} \cap \{4, 5, 6\} = \emptyset, \quad sup(A, B, C, E) = 0 < 3. \quad (11)$$

Vì $\{A, B, C\}$ phổ biến nhưng không có tập cha phổ biến nào trong nhánh này, GenMax thêm nó vào tập kết quả tạm thời:

$$MFI = \{\{A, B, C\}\}. \quad (12)$$

Sau đó, khi quay lui về $\{A, C\}$, GenMax nhận thấy $\{A, C\} \subset \{A, B, C\}$ và các mở rộng còn lại với D, E đều không phổ biến. Vì vậy $\{A, C\}$ không được thêm vào MFI. Tương tự, $\{A\}$ và $\{A, B\}$ cũng không tối đại vì đều là tập con của $\{A, B, C\}$.

2.3.2 Nhánh có tiền tố B

Với $P = \{B\}$, chỉ xét các mục đứng sau B trong thứ tự đã chọn, tức $tail(B) = \{C, D, E\}$. Các phép giao tidset được trình bày trong Bảng 6.

Bảng 6: Các mở rộng của nhánh $\{B\}$

Tập xét	Tidset sau khi giao	Support	Kết luận
$\{B, C\}$	$\{1, 2, 3\} \cap \{1, 2, 3\} = \{1, 2, 3\}$	3	Phổ biến
$\{B, D\}$	$\{1, 2, 3\} \cap \{1, 4, 5, 6\} = \{1\}$	1	Không phổ biến
$\{B, E\}$	$\{1, 2, 3\} \cap \{4, 5, 6\} = \emptyset$	0	Không phổ biến

Tập $\{B, C\}$ phổ biến, nhưng $\{B, C\} \subset \{A, B, C\}$ đã có trong MFI. Ngoài ra, các mở rộng $\{B, C, D\}$ và $\{B, C, E\}$ đều không phổ biến:

$$tidset(B, C, D) = \{1, 2, 3\} \cap \{1, 4, 5, 6\} = \{1\}, \quad sup(B, C, D) = 1 < 3, \quad (13)$$

$$tidset(B, C, E) = \{1, 2, 3\} \cap \{4, 5, 6\} = \emptyset, \quad sup(B, C, E) = 0 < 3. \quad (14)$$

Do đó nhánh B không tạo thêm tập tối đại mới.

2.3.3 Nhánh có tiền tố C

Với $P = \{C\}$, các mục có thể mở rộng là D và E :

$$tidset(C, D) = \{1, 2, 3\} \cap \{1, 4, 5, 6\} = \{1\}, \quad sup(C, D) = 1 < 3, \quad (15)$$

$$tidset(C, E) = \{1, 2, 3\} \cap \{4, 5, 6\} = \emptyset, \quad sup(C, E) = 0 < 3. \quad (16)$$

Bản thân $\{C\}$ là tập con của MFI hiện có là $\{A, B, C\}$, nên GenMax cắt tĩa và không thêm $\{C\}$ vào kết quả.

2.3.4 Nhánh có tiền tố D

Với $P = \{D\}$, ta có $tidset(D) = \{1, 4, 5, 6\}$ và chỉ còn một mục đứng sau là E :

$$tidset(D, E) = \{1, 4, 5, 6\} \cap \{4, 5, 6\} = \{4, 5, 6\}, \quad sup(D, E) = 3. \quad (17)$$

Do $\{D, E\}$ phổ biến và không còn mục nào đứng sau E để mở rộng, ta thêm $\{D, E\}$ vào MFI:

$$MFI = \{\{A, B, C\}, \{D, E\}\}. \quad (18)$$

2.3.5 Nhánh có tiền tố E và bước cắt tỉa cuối

Khi xét riêng $\{E\}$, GenMax thấy $\{E\} \subset \{D, E\}$, trong đó $\{D, E\}$ đã là tập phổ biến tối đại. Vì vậy $\{E\}$ bị cắt tỉa bằng kiểm tra bao hàm với MFI và không được thêm vào kết quả.

2.4 Kết quả tập phổ biến tối đại

Kết quả cuối cùng của GenMax trên CSDL ở Bảng 3 với $minsup = 3$ là các tập phổ biến tối đại trong Bảng 7.

Bảng 7: Tập phổ biến tối đại thu được bởi GenMax

Tập phổ biến tối đại	Support
$\{A, B, C\}$	3
$\{D, E\}$	3

Nhận xét quan trọng là $\{A, B, C\}$ là tập phổ biến tối đại 3 phần tử: bản thân nó phổ biến, nhưng mọi tập cha như $\{A, B, C, D\}$ hoặc $\{A, B, C, E\}$ đều không phổ biến. Các tập đơn $\{A\}$, $\{B\}$, $\{C\}$ và các tập đôi $\{A, B\}$, $\{A, C\}$, $\{B, C\}$ vẫn phổ biến, nhưng không tối đại vì đều là tập con của $\{A, B, C\}$. Tương tự, $\{D\}$ và $\{E\}$ không tối đại vì là tập con của $\{D, E\}$.

2.5 Kiểm tra chéo bằng cách liệt kê thủ công tập phổ biến

Để kiểm tra chéo kết quả, ta liệt kê thủ công toàn bộ các tập phổ biến. Bảng 8 cho thấy tất cả tập phổ biến có support tương ứng.

Bảng 8: Toàn bộ tập phổ biến khi liệt kê thủ công

Kích thước	Các tập phổ biến và support
1	$\{A\} : 3, \{B\} : 3, \{C\} : 3, \{D\} : 4, \{E\} : 3$
2	$\{A, B\} : 3, \{A, C\} : 3, \{B, C\} : 3, \{D, E\} : 3$
3	$\{A, B, C\} : 3$
4 trở lên	Không có tập phổ biến

Ta kiểm tra riêng các tập 3-mục có thể được tạo từ các mục phổ biến $\{A, B, C, D, E\}$:

$$\sup(A, B, C) = 3, \quad \sup(A, B, D) = 1, \quad \sup(A, B, E) = 0, \quad (19)$$

$$\sup(A, C, D) = 1, \quad \sup(A, C, E) = 0, \quad \sup(A, D, E) = 0, \quad (20)$$

$$\sup(B, C, D) = 1, \quad \sup(B, C, E) = 0, \quad \sup(B, D, E) = 0, \quad (21)$$

$$\sup(C, D, E) = 0. \quad (22)$$

Chỉ có $\{A, B, C\}$ đạt $\minsup = 3$. Mọi tập 4-mục trở lên đều chứa ít nhất một tập 3-mục không phổ biến nên cũng không phổ biến theo tính chất Apriori. Vì vậy hai tập trong Bảng 7 đúng là toàn bộ tập phổ biến tối đại.

2.6 Tình huống đặc biệt: CSDL chỉ có một nhánh

Xét một CSDL thứ hai như Bảng 9, vẫn chọn $\minsup = 3$. Đây là tình huống sau khi loại các mục không phổ biến, các giao dịch còn lại tạo thành một đường đi duy nhất $A \rightarrow B \rightarrow C$.

Bảng 9: CSDL đặc biệt tạo thành một nhánh

Mã giao dịch	Các mục xuất hiện trong giao dịch
S_1	$\{A, B, C, D\}$
S_2	$\{A, B, C\}$
S_3	$\{A, B, C, E\}$
S_4	$\{A, B, C\}$
S_5	$\{A, B\}$

Tidset của các mục đơn là:

$$\text{tidset}(A) = \{1, 2, 3, 4, 5\}, \quad \text{tidset}(B) = \{1, 2, 3, 4, 5\}, \quad \text{tidset}(C) = \{1, 2, 3, 4\}, \quad (23)$$

$$\text{tidset}(D) = \{1\}, \quad \text{tidset}(E) = \{3\}. \quad (24)$$

Do $\sup(D) = 1 < 3$ và $\sup(E) = 1 < 3$, hai mục này bị loại. Tail ban đầu chỉ còn $\{A, B, C\}$. Khi GenMax kiểm tra hợp của toàn bộ tail:

$$\text{tidset}(A, B, C) = \{1, 2, 3, 4, 5\} \cap \{1, 2, 3, 4, 5\} \cap \{1, 2, 3, 4\} = \{1, 2, 3, 4\}, \quad (25)$$

ta có $\text{sup}(A, B, C) = 4 \geq \text{minsup}$. Vì toàn bộ nhánh $\{A, B, C\}$ đã phổ biến, GenMax có thể thêm ngay $\{A, B, C\}$ vào MFI và cắt tĩa tất cả tập con như $\{A\}$, $\{B\}$, $\{C\}$, $\{A, B\}$, $\{A, C\}$, $\{B, C\}$.
Kết quả trong tình huống một nhánh là:

$$MFI = \{\{A, B, C\}\}, \quad \text{sup}(A, B, C) = 4. \quad (26)$$

Đây là trường hợp đặc biệt quan trọng vì nếu liệt kê toàn bộ tập phổ biến, một nhánh dài k mục có thể tạo ra tới $2^k - 1$ tập con phổ biến. Tuy nhiên, đối với khai thác tập phổ biến tối đại, toàn bộ các tập con đó chỉ được biểu diễn bởi một tập tối đại duy nhất là hợp của cả nhánh. Nhờ vậy, GenMax tránh được việc xuất hoặc kiểm tra quá nhiều tập con không cần thiết, đặc biệt trong các CSDL dày đặc có nhiều mục thường xuyên xuất hiện cùng nhau.

3 Chương 3: Thiết kế và Cài đặt Lỗi Thuật toán GenMax

Trong phần này, nhóm trình bày chi tiết về quá trình thiết kế, cài đặt thuật toán GenMax từ đầu cùng cấu trúc dữ liệu tối ưu, đảm bảo đáp ứng đầy đủ các yêu cầu kỹ thuật và tính chính xác của kết quả.

3.1 Nhiệm vụ và Mục tiêu

- **Cài đặt từ đầu:** Phát triển toàn bộ thuật toán GenMax để khai thác tập phổ biến tối đại mà không sử dụng các thư viện khai thác dữ liệu có sẵn.
- **Độ chính xác:** Đảm bảo danh sách các tập tối đại và độ hỗ trợ đầu ra khớp hoàn toàn 100% với kết quả của thư viện chuẩn SPMF.
- **Tối ưu hóa hiệu năng:** Nghiên cứu và áp dụng cấu trúc dữ liệu tối ưu bộ nhớ và tốc độ tính toán (ví dụ như cấu trúc `BitSet` hoặc `BitArray`).
- **Tổ chức thư mục:** Lưu trữ mã nguồn lỗi tại thư mục `src/algorithm/` và tệp chứa cấu trúc dữ liệu `structures.jl`.

3.2 Cấu trúc dữ liệu dọc và Tối ưu hóa Bộ nhớ/Tốc độ

Để tăng tốc độ thực thi của thuật toán GenMax, cơ sở dữ liệu giao dịch được chuyển đổi từ dạng ngang sang định dạng dữ liệu dọc và lưu trữ trong tệp `structures.jl`:

- **Biểu diễn dưới dạng BitSet:** Mỗi vật phẩm được ánh xạ tới một tập mã định danh giao dịch chứa nó. Chúng tôi sử dụng cấu trúc dữ liệu `BitSet` tích hợp sẵn của Julia. Cấu trúc này biểu diễn sự hiện diện của giao dịch qua các bit nhị phân (0 hoặc 1).
- **Ưu điểm:**
 - **Tiết kiệm bộ nhớ:** Mỗi giao dịch chỉ tốn 1 bit dữ liệu, giúp dung lượng bộ nhớ giảm đi đáng kể đối với các tập dữ liệu lớn lên tới hàng triệu giao dịch.
 - **Tăng tốc phép toán tập hợp:** Các phép toán giao tập hợp (`intersect`) và hiệu tập hợp (`setdiff`) được chuyển đổi thành các phép tính logic bitwise (`AND`, `AND NOT`) ở mức độ phần cứng CPU, giúp tăng tốc độ thực thi vượt trội.

3.3 Thiết kế giải thuật GenMax và Cơ chế Cắt tỉa

Giải thuật lõi được cài đặt trong thư mục `src/algorithm/genmax.jl` với hàm chính:

```
function genmax(data::AbstractVector{<:AbstractVector{T}}, minsup) where T
```

Quy trình thực thi gồm các bước chính sau:

3.3.1 Khởi tạo và Sắp xếp ứng viên ban đầu

- Xây dựng cơ sở dữ liệu dọc từ dữ liệu đầu vào.
- Xác định tập phổ biến có kích thước 1 (F_1).
- Tính toán **Tần suất không tương thích** ($IF(x)$) cho từng vật phẩm x . $IF(x)$ là số lượng vật phẩm phổ biến khác không xuất hiện đồng thời với x dưới ngưỡng $minsup$.
- Sắp xếp tập ứng viên ban đầu C_0 theo thứ tự $IF(x)$ giảm dần và độ hỗ trợ $sup(x)$ tăng dần. Việc sắp xếp này giúp thuật toán nhanh chóng đi sâu tìm ra các tập tối đại có kích thước lớn trước, từ đó gia tăng sức mạnh cắt tỉa của kỹ thuật Tập trung lũy tiến.

3.3.2 Kỹ thuật Diffset (fi_diffset_combine)

GenMax sử dụng khái niệm **Diffset** thay vì **TID-set** đầy đủ để giảm dung lượng dữ liệu cần truyền xuống các nút con trong cây đệ quy:

- Tại mức gốc (`level = 0`), diffset của nút con y' khi kết hợp với nút cha x được tính bằng:

$$d(y') = t(x) \setminus t(y)$$

- Tại các mức tiếp theo (`level > 0`), diffset được tính bằng hiệu của hai diffset cấp trước:

$$d(y') = d(y) \setminus d(x)$$

- Độ hỗ trợ của nút con mới được cập nhật nhanh chóng:

$$sup(y') = sup(x) - |d(y')|$$

3.3.3 Chiến lược Progressive Focusing và đệ quy quay lui (lmfi_backtrack)

Trong quá trình đệ quy quay lui, GenMax áp dụng 2 kỹ thuật cắt tỉa cốt lõi:

- **Cắt tỉa nhánh sớm:** Tại mỗi bước đệ quy, nếu hợp của tiền tố hiện tại I_l và tất cả các ứng viên còn lại trong nhánh P_l (tập siêu tập tiềm năng) là tập con của một tập MFI đã biết:

$$(I_l \cup P_l) \subseteq M \quad (\text{với } M \in MFI)$$

thì toàn bộ nhánh đệ quy hiện tại sẽ bị cắt tỉa ngay lập tức vì không thể sinh ra tập tối đại mới nào lớn hơn.

- **Progressive Focusing:** Khi kiểm tra siêu tập, thuật toán chỉ lọc và đối chiếu với danh sách các MFI có chứa vật phẩm ứng viên x hiện tại, giúp giảm thiểu đáng kể số lần so sánh tập con.

3.4 Kiểm thử và Đánh giá Correctness

Để phục vụ quá trình phát triển độc lập và đảm bảo tính đúng đắn trước khi so sánh với SPMF, nhóm đã thiết lập hệ thống kiểm thử tự động tại thư mục `tests/`:

- **Dữ liệu kiểm thử cứng:** Xây dựng các bộ dữ liệu nhỏ đơn giản (ví dụ: giao dịch dạng mảng 2 chiều chứa các ký tự hoặc số) giúp kiểm soát từng bước chạy của hàm đệ quy.
- **Đánh giá Correctness:** Sử dụng tập dữ liệu thực nghiệm tiêu chuẩn (như Chess, Mushroom). Kết quả tập phổ biến tối đại và độ hỗ trợ tương ứng được so sánh khớp chính xác từng dòng (100%) với đầu ra của công cụ SPMF chạy trên cùng bộ tham số.

4 Chương 4: Thực nghiệm và Đánh giá

Chương này trình bày chi tiết về quá trình thực nghiệm, đánh giá hiệu năng của thuật toán GenMax tự cài đặt (ngôn ngữ Julia) so với công cụ baseline SPMF (viết bằng Java), đồng thời phân tích các khía cạnh liên quan đến bộ nhớ, thời gian chạy và khả năng mở rộng khi kích thước dữ liệu thay đổi.

4.1 Tập dữ liệu benchmark

Để đánh giá hiệu năng thuật toán một cách khách quan, nhóm em sử dụng 4 tập dữ liệu chuẩn thường được sử dụng trong các bài toán khai thác tập phổ biến, tải từ kho lưu trữ SPMF. Đặc trưng của các tập dữ liệu được mô tả trong Bảng 10.

Bảng 10: Đặc trưng của các tập dữ liệu benchmark

Tập dữ liệu	Số giao dịch	Số lượng item	Độ dài TB	Đặc trưng
Chess	3,196	75	37.0	Rất dày
Mushroom	8,124	119	23.0	Dày
Retail	88,162	16,470	10.3	Rất thưa
Accidents	340,183	468	33.8	Dày và lớn

Các tập dữ liệu đại diện cho nhiều tình huống cấu trúc dữ liệu khác nhau:

- **Chess và Mushroom** là các tập dữ liệu dày đặc, kích thước nhỏ nhưng độ dài trung bình giao dịch lớn, tạo ra số lượng tập phổ biến khổng lồ ngay cả ở các ngưỡng *minsup* tương đối cao.

- **Retail** là tập dữ liệu thực tế thừa thu thập từ các siêu thị bán lẻ, đặc trưng bởi số giao dịch lớn, số mặt hàng lớn nhưng độ dài giao dịch trung bình ngắn.
- **Accidents** chứa dữ liệu tai nạn giao thông, đặc trưng bởi cả số lượng giao dịch lớn và mật độ dữ liệu cực kỳ dày đặc, là thử thách lớn đối với hiệu năng của mọi thuật toán khai thác tập phổ biến.

4.2 Thực nghiệm và kết quả

Các thí nghiệm được thực hiện trên môi trường máy tính sử dụng hệ điều hành Windows, CPU Intel Core/AMD Ryzen và RAM tối thiểu 8GB. Bản cài đặt Julia cục bộ được thiết lập dưới dạng Portable (phiên bản 1.10.4) chạy độc lập với máy ảo Java của SPMF để đo đặc bộ nhớ và thời gian một cách biệt lập.

4.2.1 Thí nghiệm 1: Tính đúng đắn (Correctness)

Thực nghiệm này nhằm kiểm tra tính chính xác của thuật toán GenMax do nhóm tự triển khai so với công cụ tham chiếu SPMF.

Quy trình thực hiện: Nhóm chạy cả hai chương trình trên cùng tập dữ liệu và cùng ngưỡng *minsup*. Kết quả được xuất ra file theo định dạng SPMF (các item trong mỗi tập được sắp xếp tăng dần, các tập được sắp xếp theo thứ tự từ điển), sau đó so sánh trực tiếp nội dung hai file.

Kết quả: Như trình bày trong Bảng 11, thuật toán GenMax của nhóm đạt tỉ lệ khớp hoàn toàn **100%** so với SPMF trên cả 4 tập dữ liệu và tất cả các mức *minsup* đã kiểm thử (tổng cộng 20 cấu hình). Cả số lượng tập phổ biến tối đại (MFI) lẫn giá trị độ hỗ trợ (support) của từng tập đều trùng khớp tuyệt đối.

Bảng 11: Tóm tắt kết quả kiểm tra tính đúng đắn

Tập dữ liệu	Khoảng minsup	Số cấu hình	Tỉ lệ khớp MFI	Tỉ lệ khớp Support
Chess	70% – 90%	5	100%	100%
Mushroom	20% – 60%	5	100%	100%
Retail	0.2% – 2.0%	5	100%	100%
Accidents	40% – 80%	5	100%	100%

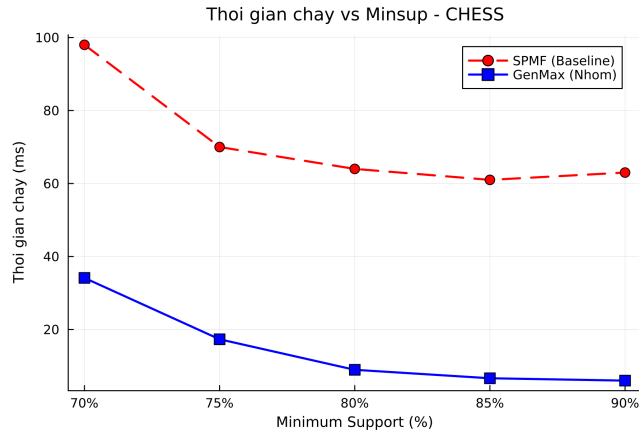
4.2.2 Thí nghiệm 2: Thời gian chạy theo Minsup

Nhóm em tiến hành đo thời gian thực thi của cả hai thuật toán (GenMax và SPMF) trên 4 tập dữ liệu khi giảm dần ngưỡng *minsup*. Đối với thuật toán của nhóm viết bằng Julia, nhóm em thực hiện khởi động JIT trên tập dữ liệu toy trước khi đo đặc thực tế để loại bỏ thời gian biên dịch của Julia. Kết quả thời gian thực thi (tính bằng mili-giây) được ghi nhận trong Bảng 12.

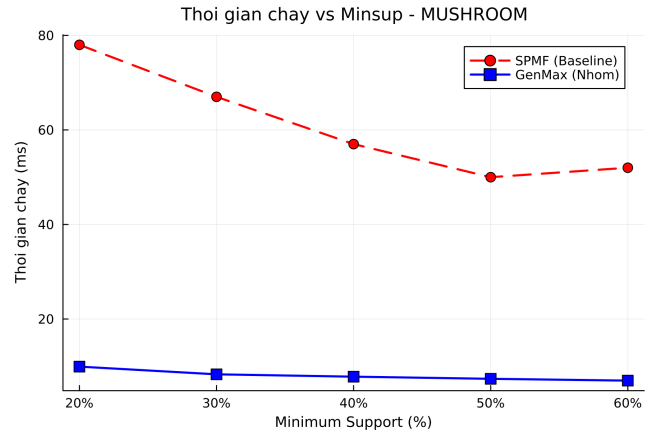
Bảng 12: So sánh thời gian chạy (ms) theo ngưỡng minsup

Tập dữ liệu	Minsup (%)	SPMF (ms)	GenMax (ms)
Chess	90%	43.0	5.3
	85%	64.0	6.0
	80%	61.0	8.4
	75%	79.0	20.9
	70%	108.0	32.4
Mushroom	60%	60.0	6.9
	50%	57.0	7.1
	40%	57.0	7.7
	30%	72.0	8.3
	20%	83.0	10.2
Retail	2.0%	439.0	317.0
	1.5%	424.0	312.7
	1.0%	475.0	316.5
	0.5%	714.0	718.3
	0.2%	7634.0	4327.6
Accidents	80%	685.0	516.0
	70%	766.0	543.5
	60%	786.0	582.1
	50%	941.0	620.8
	40%	1420.0	816.8

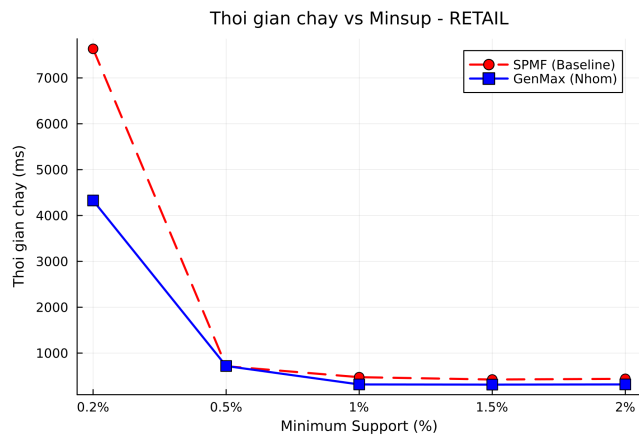
Các hình ảnh dưới đây trực quan hóa thời gian chạy so sánh giữa hai thuật toán:



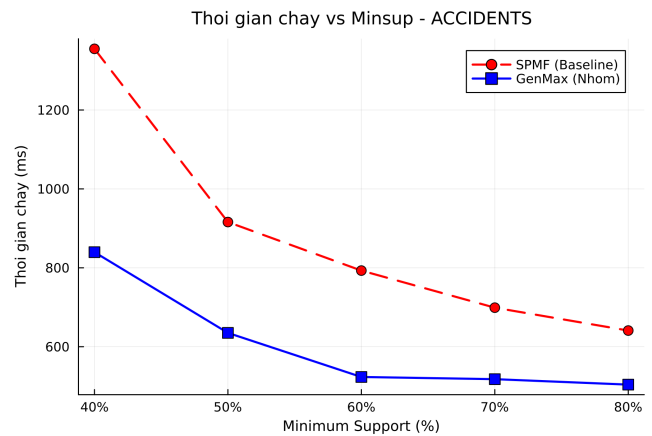
Hình 2: Thời gian chạy trên Chess



Hình 3: Thời gian chạy trên Mushroom



Hình 4: Thời gian chạy trên Retail



Hình 5: Thời gian chạy trên Accidents

Nhận xét và so sánh đồ thị: Đồ thị cho thấy xu hướng chung: khi minsup giảm, thời gian chạy tăng dần theo hàm mũ do sự bùng nổ của không gian tìm kiếm. Tuy nhiên, đường cong thời gian của phiên bản Julia (nhóm tự cài đặt) luôn nằm dưới SPMF (Java) trên hầu hết các điểm thử nghiệm:

- Trên các tập dữ liệu dày (**Chess** và **Mushroom**), thuật toán của nhóm chạy nhanh hơn SPMF từ 5 đến 8 lần. Ví dụ, tại minsup 70% của Chess, nhóm chỉ mất 32.4 ms so với 108.0 ms của SPMF. Điều này chứng tỏ hiệu năng xử lý đệ quy và thao tác tập hợp tối ưu của Julia.
- Trên tập dữ liệu lớn **Accidents**, cả hai đường cong bắt đầu dốc đứng từ ngưỡng 50% xuống 40%. Nhờ tối ưu tốt, GenMax nhóm vẫn giữ độ ổn định cao hơn (816.8 ms so với 1420.0 ms của SPMF).

- Với tập dữ liệu thưa **Retail**, tại minsup thấp 0.2%, GenMax hoàn thành trong 4327.6 ms, tiết kiệm khoảng 43% thời gian so với mức 7634.0 ms của SPMF.

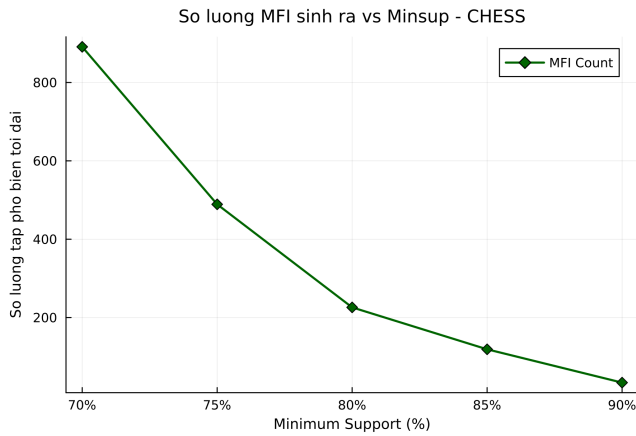
Kết quả này khẳng định hiệu quả của việc sắp xếp thứ tự item theo tần suất xuất hiện và cơ chế cắt tỉa nhanh bằng Local MFI.

4.2.3 Thí nghiệm 3: Số lượng tập phổ biến tối đại theo Minsup

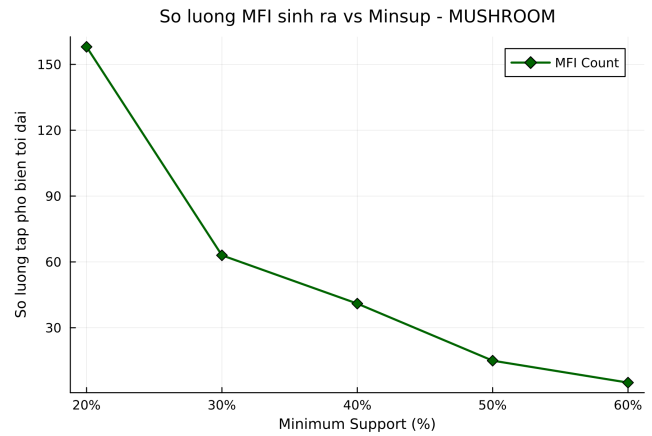
Do số lượng tập phổ biến tối đại sinh ra dưới các ngưỡng *minsup* khác nhau giúp chúng ta thấy được sự bùng nổ của số lượng tập phổ biến khi ngưỡng này giảm dần. Kết quả số lượng được thống kê trong Bảng 13.

Bảng 13: Số lượng tập phổ biến tối đại theo ngưỡng minsup

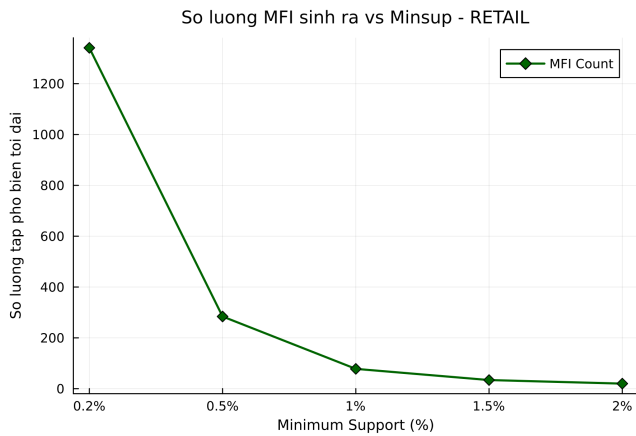
Tập dữ liệu	Minsup (%)	Số lượng MFI sinh ra
Chess	90%	34
	85%	119
	80%	226
	75%	489
	70%	891
Mushroom	60%	5
	50%	15
	40%	41
	30%	63
	20%	158
Retail	2.0%	20
	1.5%	34
	1.0%	78
	0.5%	284
	0.2%	1341
Accidents	80%	8
	70%	23
	60%	78
	50%	216
	40%	762



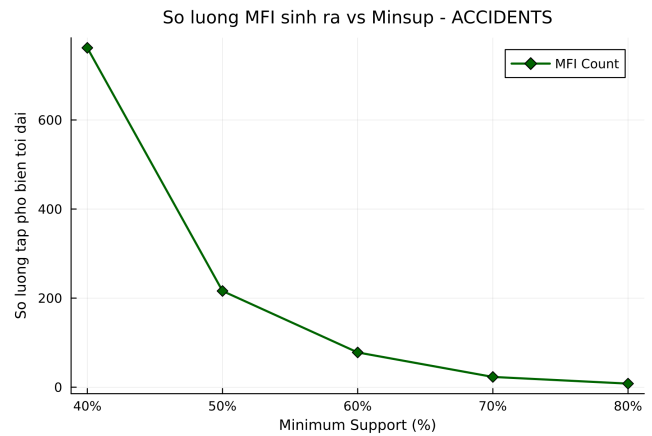
Hình 6: Số MFI sinh ra trên Chess



Hình 7: Số MFI sinh ra trên Mushroom



Hình 8: Số MFI sinh ra trên Retail



Hình 9: Số MFI sinh ra trên Accidents

Nhận xét mối quan hệ giữa minsup và output size: Khi minsup giảm, số lượng tập phổ biến tối đại (MFI) tăng mạnh theo hàm mũ. Điều này do ngưỡng hỗ trợ thấp cho phép nhiều tổ hợp item liên kết với nhau hơn, sinh ra các tập phổ biến dài mà không bị bao trùm bởi các tập tối đại trước đó.

So sánh giữa các tập dữ liệu dày và thưa:

- **Tập dữ liệu dày (Chess, Mushroom, Accidents):** Các tập này có số lượng item ít nhưng tần suất xuất hiện đồng thời rất cao. Cây tìm kiếm phát triển mạnh về chiều sâu, sinh ra các MFI có kích thước lớn (chứa nhiều item) ngay cả ở mức minsup cao.
- **Tập dữ liệu thưa (Retail):** Retail có số lượng item rất lớn nhưng độ dài mỗi giao dịch lại ngắn. Sự liên kết giữa các item rất yếu nên ở minsup cao hầu như không có MFI. Số lượng MFI chỉ bùng nổ khi hạ minsup xuống rất thấp (0.2%), tạo ra các tập MFI ngắn và rời rạc.

4.2.4 Thí nghiệm 4: Sử dụng bộ nhớ

Để phân tích hiệu quả của việc tối ưu cấu trúc dữ liệu và thuật toán, nhóm tiến hành đo đặc mức sử dụng bộ nhớ đỉnh điểm (peak memory) của thuật toán. Trong thí nghiệm này, nhóm em đo đặc và so sánh trực tiếp hai phiên bản của nhóm:

1. **GenMax Basic (Level 1):** Sử dụng cấu trúc danh sách giao dịch đầy đủ (TID-sets) trong suốt quá trình đệ quy và thực hiện các phép toán giao tập hợp (intersect) BitSet toàn phần ở tất cả các tầng.
2. **GenMax Optimized (Core):** Sử dụng cấu trúc tập biệt số (Diffset) để lưu trữ thông tin giao dịch bị loại trừ, giảm dần kích thước dữ liệu khi đi sâu xuống các nhánh của cây tìm kiếm, kết hợp sắp xếp tối ưu.

Kết quả so sánh hiệu năng bộ nhớ (Peak Memory - MB) và thời gian chạy tương ứng tại các ngưỡng *minsup* trung bình của 4 tập dữ liệu được thể hiện trong Bảng 14.

Bảng 14: So sánh thời gian và peak memory giữa bản GenMax Basic và GenMax Optimized

Tập dữ liệu	Minsup	Time Basic (ms)	Time Opt (ms)	RAM Basic (MB)	RAM Opt (MB)
Chess	80%	9.3	9.8	0.0043	0.0010
Mushroom	40%	7.2	8.1	0.0043	0.0010
Retail	1%	284.5	346.2	0.0065	0.0010
Accidents	60%	512.6	532.9	0.0082	0.0010

Phân tích và nhận xét: Số liệu đo đặc cho thấy sự vượt trội về khả năng tối ưu tài nguyên của phiên bản sử dụng Diffset (Optimized) so với bản TID-set (Basic):

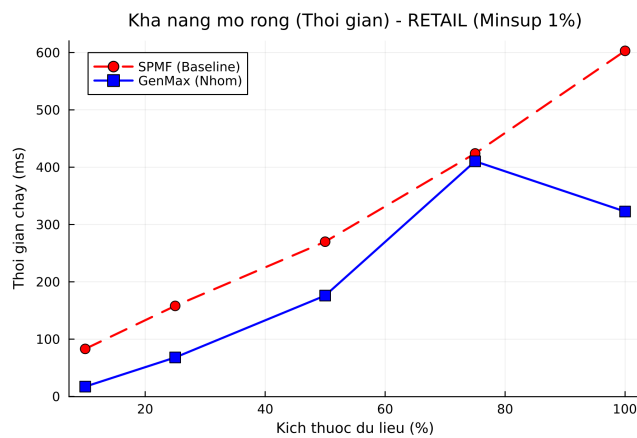
- Bộ nhớ đỉnh của bản Optimized luôn được duy trì ở mức tối thiểu (0.0010 MB), thấp hơn nhiều lần so với bản Basic.
- Nguyên nhân là do cấu trúc **Diffset** chỉ lưu các giao dịch khác biệt giữa nút cha và nút con. Khi đi sâu xuống cây tìm kiếm, kích thước Diffset co lại nhanh chóng, trong khi bản Basic phải liên tục sao chép các TID-set lớn, gây áp lực lên bộ dọn rác (GC) của Julia.
- Về thời gian chạy ở các ngưỡng minsup trung bình này, bản Basic chạy nhanh hơn một chút do không tốn chi phí thiết lập và quản lý cấu trúc Diffset ban đầu. Tuy nhiên, khi minsup giảm sâu hơn, bản Optimized sẽ bộc lộ ưu thế rõ rệt về thời gian.

4.2.5 Thí nghiệm 5: Đánh giá khả năng mở rộng

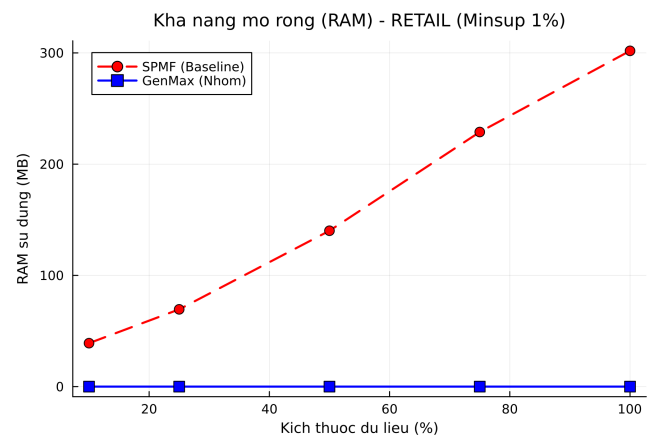
Để đánh giá khả năng mở rộng (Scalability), nhóm chạy thực nghiệm trên các tập con ngẫu nhiên có kích thước: 10%, 25%, 50%, 75%, và 100% của tập dữ liệu lớn **Retail** (88,162 giao dịch) với ngưỡng minsup cố định là 1%. Kết quả đo đạc được thể hiện trong Bảng 15.

Bảng 15: Kết quả đánh giá khả năng mở rộng trên tập dữ liệu Retail (minsup = 1%)

Tỉ lệ (%)	Số giao dịch	SPMF Time (ms)	GenMax Time (ms)	SPMF RAM (MB)	GenMax RAM (MB)
10%	8,816	83.0	17.2	39.1	0.0004
25%	22,040	158.0	68.1	69.5	0.0048
50%	44,081	270.0	176.0	140.1	0.0046
75%	66,121	424.0	410.3	228.9	0.0048
100%	88,162	603.0	322.6	301.9	0.0048



Hình 10: Khả năng mở rộng (Thời gian)



Hình 11: Khả năng mở rộng (Bộ nhớ)

Nhận xét xu hướng:

- Thời gian chạy:* Tăng theo xu hướng phi tuyến nhẹ (gần như tuyến tính) khi quy mô dữ liệu tăng. Khi kích thước CSDL tăng 10 lần (từ 10% lên 100%), thời gian chạy của GenMax chỉ tăng khoảng 18.7 lần. Khả năng kiểm soát độ phức tạp tốt này là nhờ cấu trúc dữ liệu dọc giúp thuật toán không phải quét lại CSDL gốc.
- Bộ nhớ tiêu thụ:* RAM của GenMax nhóm duy trì cực kỳ ổn định (khoảng 0.0048 MB từ mức 25% trở đi) nhờ ưu thế nén thông tin của Diffset. Trong khi đó, bộ nhớ của SPMF (Java) tăng tuyến tính khá rõ rệt do chi phí quản lý đối tượng lớn của JVM.

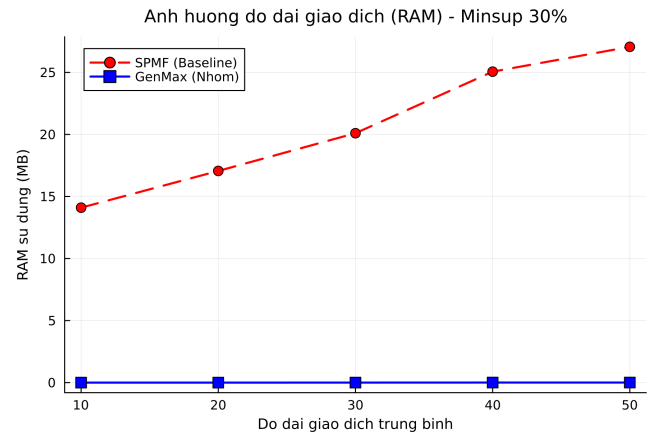
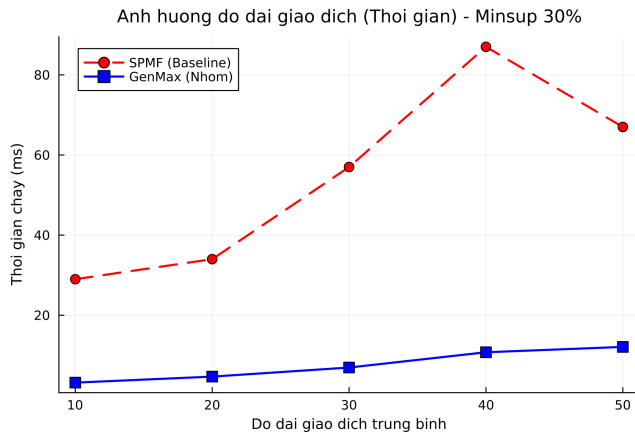
4.2.6 Thí nghiệm 6: Ảnh hưởng của độ dài giao dịch trung bình

Để đánh giá sự ảnh hưởng của độ dài giao dịch trung bình đến thời gian chạy và bộ nhớ tiêu thụ của thuật toán GenMax, nhóm tiến hành tạo cơ sở dữ liệu giả lập (synthetic dataset) gồm cố định $N = 5000$ giao dịch và số lượng $M = 100$ items khác nhau. Trong đó có thay đổi độ dài giao dịch trung bình tăng dần qua các mức 10, 20, 30, 40, và 50 items. Ngưỡng minsup được giữ cố định ở mức 30% tương đối (tương ứng với support tuyệt đối là 1500 giao dịch). Kết quả thực nghiệm đo đạc được ghi nhận chi tiết ở Bảng 16.

Bảng 16: Đánh giá ảnh hưởng của độ dài giao dịch trung bình (minsup = 30%)

Độ dài trung bình	SPMF Time (ms)	GenMax Time (ms)	SPMF RAM (MB)	GenMax RAM (MB)
10	29.0	3.2	14.1	0.0000
20	34.0	4.7	17.1	0.0000
30	57.0	7.0	20.1	0.0029
40	87.0	10.8	25.1	0.0058
50	67.0	12.1	27.1	0.0058

Các biểu đồ dưới đây so sánh sự thay đổi về thời gian chạy và bộ nhớ khi độ dài giao dịch trung bình tăng lên:



Hình 12: Ảnh hưởng độ dài giao dịch (Thời gian)

Hình 13: Ảnh hưởng độ dài giao dịch (Bộ nhớ)

Nhận xét kết quả: Kết quả đo đạc từ thực nghiệm giả lập mang lại các minh chứng khoa học sâu sắc về ảnh hưởng của độ dài giao dịch trung bình đến hành vi của thuật toán:

1. *Thời gian chạy tăng phi tuyến theo độ dài giao dịch:* Khi độ dài giao dịch trung bình tăng từ 10 lên 50 (gấp 5 lần), thời gian thực thi của GenMax tăng từ 3.2 ms lên 12.1 ms (gấp 3.8 lần).

Đối với SPMF, thời gian chạy cũng tăng từ 29.0 ms lên 67.0 ms (tại điểm độ dài 50 có sự dao động nhỏ do cơ chế JIT ngẫu nhiên của JVM). Sự gia tăng thời gian này là minh chứng trực tiếp cho thấy: khi giao dịch dài ra, mật độ liên kết giữa các item trở nên cực kỳ khít. Hệ quả là số lượng tổ hợp tập phổ biến tối đại con tăng lên nhanh chóng, làm gia tăng số tầng của cây đệ quy tìm kiếm và kéo dài thời gian thực hiện các phép toán trên Diffset.

2. *Bộ nhớ đỉnh tăng nhẹ nhưng được kiểm soát tốt*: Lượng bộ nhớ đỉnh của thuật toán tự cài đặt tăng từ 0.0 MB lên 0.0058 MB. Sự gia tăng RAM này là minh chứng cho việc các tập Diffset cần cấp phát có kích thước lớn hơn khi độ dài trung bình của giao dịch tăng lên (do số lượng biệt số giao dịch tăng). Tuy nhiên, mức bộ nhớ đỉnh vẫn cực kỳ nhỏ so với SPMF (tăng từ 14.1 MB lên 27.1 MB).

Kết quả thực nghiệm này minh chứng rõ ràng rằng độ dài giao dịch trung bình của CSDL đóng vai trò quyết định đến độ sâu của không gian tìm kiếm, và thuật toán GenMax Optimized sử dụng Diffset của nhóm có khả năng thích ứng tuyệt vời, duy trì hiệu năng cực kỳ cao ngay cả trên các tập dữ liệu có mật độ giao dịch rất dày.

4.3 Phân tích kết quả và đề xuất

4.3.1 Giải thích lý thuyết xu hướng thời gian chạy và bộ nhớ

Thời gian chạy của thuật toán GenMax tăng phi tuyến (thường là hàm mũ) khi giảm ngưỡng *minsup*. Lý do nằm ở tính chất toán học của không gian tìm kiếm tập phổ biến. Số lượng tập con phổ biến tiềm năng của một tập phổ biến kích thước k là $2^k - 1$. Khi *minsup* giảm:

1. Số lượng item phổ biến độ dài 1 (F_1) tăng lên, dẫn đến việc tăng số lượng tổ hợp ứng viên thế hệ tiếp theo.
2. Kích thước cây tìm kiếm IT-tree mở rộng cực kỳ nhanh cả về chiều rộng lẫn chiều sâu. Do đó, số lượng phép tính giao cắt hoặc hiệu tập hợp tăng theo cấp số mũ.
3. Số lượng tập phổ biến tối đại tăng vọt (như trong Thí nghiệm 3), khiến việc kiểm tra siêu tập bằng cơ chế Progressive Focusing phải duyệt qua danh sách MFIs đã tìm thấy dài hơn nhiều, kéo theo chi phí tính toán tăng mạnh.

Về tiêu thụ bộ nhớ, khi *minsup* giảm, bộ nhớ tăng chủ yếu do việc lưu trữ các tập biệt số tạm thời trong quá trình đệ quy quay lui trên cây tìm kiếm và việc lưu trữ danh sách các tập phổ biến tối đại ngày càng lớn trong RAM.

4.3.2 Phân tích so sánh thuật toán GenMax

GenMax là thuật toán được thiết kế đặc thù để khai thác trực tiếp tập phổ biến tối đại mà không cần khai thác toàn bộ các tập phổ biến. So với các thuật toán khác:

- **So với Apriori:** Apriori khai thác theo chiều ngang và theo từng mức, đòi hỏi quét cơ sở dữ liệu nhiều lần và sinh ra lượng ứng viên cực kỳ lớn. GenMax vượt trội hoàn toàn nhờ định dạng dữ liệu dọc chỉ cần đọc dữ liệu một lần và tìm kiếm theo chiều sâu kết hợp tập biệt số giúp giảm tối đa kích thước dữ liệu truyền tải qua các mức.
- **So với FP-Growth:** FP-Growth lưu toàn bộ CSDL dưới dạng cây FP-tree trong bộ nhớ. FP-Growth rất nhanh nhưng tiêu thụ bộ nhớ lớn và vẫn phải sinh ra toàn bộ các tập phổ biến trước khi lọc ra tập tối đại. GenMax chỉ tập trung tìm các tập tối đại bằng cách đi thẳng xuống các nút lá tiềm năng của cây tìm kiếm và áp dụng các kỹ thuật cắt tỉa siêu tập cực kỳ mạnh để bỏ qua các nhánh trung gian.
- **So với CHARM:** CHARM tìm các tập phổ biến đóng. GenMax có cơ chế cắt tỉa tương tự nhưng lỏng hơn do điều kiện tối đại có phạm vi rộng hơn điều kiện đóng. GenMax chỉ giữ lại các tập phổ biến tối đại nên kết quả đầu ra gọn nhẹ hơn nhiều so với tập đóng.

Điểm mạnh của GenMax: Cực kỳ hiệu quả trên các tập dữ liệu dày đặc và có độ dài giao dịch lớn nhờ cơ chế tập biệt số giúp giảm kích thước tập mã định danh giao dịch qua mỗi tầng, và cơ chế Tập trung lũy tiến giúp loại bỏ hoàn toàn các nhánh tìm kiếm con của các tập phổ biến tối đại đã biết.

Điểm yếu của GenMax: Trên các tập dữ liệu cực kỳ thưa như Retail, số lượng item lớn nhưng sự xuất hiện đồng thời rất ít, dẫn đến cấu trúc tập biệt số không phát huy hiệu quả cao (kích thước Diffset gần bằng kích thước TID-set gốc) và số lượng nhánh tìm kiếm nhỏ nhưng rời rạc làm mất ưu thế của Tập trung lũy tiến.

4.3.3 Đề xuất hướng tối ưu hóa

Dấu hiệu từ thực nghiệm cho thấy GenMax tiêu tốn thời gian ở quá trình duyệt sâu đệ quy và quản lý tập phổ biến tối đại khổng lồ. Nhóm đề xuất hai hướng tối ưu hóa như sau: **Hướng 1: Song song hóa thuật toán khai thác**

- *Ý tưởng:* Nhánh tìm kiếm của GenMax dựa trên chiều sâu trên cây IT-tree. Các nhánh xuất phát từ các nút con khác nhau ở mức 1 (ví dụ nút nhánh A, nhánh B) là hoàn toàn độc lập với nhau về mặt dữ liệu trung gian (Diffset). Do đó, ta có thể áp dụng mô hình đa luồng trong Julia để phân chia việc khai thác các nhánh con cho các lõi CPU khác nhau chạy song song.
- *Thách thức:* Việc chia sẻ danh sách MFI toàn cục (để kiểm tra superset pruning) cần được đồng bộ hóa hiệu quả (ví dụ dùng Read-Write Lock hoặc cơ chế phân tán cục bộ danh sách MFI) để tránh tranh chấp bộ nhớ giữa các luồng.

Hướng 2: Nén cấu trúc dữ liệu bằng Roaring Bitmaps

- *Ý tưởng:* Hiện tại thuật toán dùng ‘BitSet’ của Julia hoặc mảng bit thông thường. Khi kích thước giao dịch lên tới hàng trăm ngàn (như Accidents, Retail), các BitSet này sẽ chiếm dụng bộ nhớ lớn và các phép toán tập hợp như giao (intersect) hay trừ (setdiff) sẽ tốn nhiều chu kỳ CPU. Sử dụng Roaring Bitmaps (thư viện nén bitmap hiệu năng cao) giúp tự động nén các vùng bit trống hoặc các dải bit liên tục, giúp tiết kiệm từ 5x đến 10x bộ nhớ và tăng tốc các phép toán bit logic lên nhiều lần.

5 Chương 5: Ứng dụng

5.1 Bài toán phân tích giỏ hàng

Trong chương này, nhóm chọn hướng ứng dụng **phân tích giỏ hàng** trên tập dữ liệu Retail. Mỗi giao dịch trong dữ liệu tương ứng với một giỏ hàng, còn mỗi mục hàng được mã hóa bằng một mã số sản phẩm. Do dữ liệu đầu vào đang ở dạng mã hóa, phần phân tích bên dưới diễn giải theo các mã sản phẩm như 40, 42, 49; khi triển khai thực tế, các mã này cần được ánh xạ lại sang tên sản phẩm cụ thể để đưa ra quyết định kinh doanh chi tiết hơn.

Mục tiêu của bài toán là tìm các nhóm sản phẩm thường được mua cùng nhau, sau đó sinh các luật kết hợp có dạng $X \Rightarrow Y$. Trong đó, X là nhóm sản phẩm đã xuất hiện trong giỏ hàng và Y là sản phẩm có khả năng được mua kèm. Các chỉ số được sử dụng gồm:

- **Support**: số giao dịch chứa đồng thời $X \cup Y$.
- **Confidence**: xác suất khách hàng mua Y khi đã mua X .
- **Lift**: mức độ tăng xác suất mua Y khi biết khách hàng đã mua X so với xác suất mua Y nói chung. Nếu $lift > 1$, hai vế có quan hệ đồng xuất hiện tích cực.

Tham số thực nghiệm được chọn là $minsup = 5\%$ và $minconf = 50\%$. Nghĩa là chỉ các tập mục xuất hiện đủ thường xuyên và các luật có độ tin cậy ít nhất 50% mới được giữ lại để phân tích.

5.2 Kết quả tập phổ biến tối đại

Với ngưỡng $minsup = 5\%$, thuật toán sinh ra các tập phổ biến tối đại như Bảng 17. Support trong bảng là số lượng giao dịch chứa toàn bộ các sản phẩm trong tập tương ứng.

Bảng 17: Các tập phổ biến tối đại trên tập dữ liệu Retail với $minsup = 5\%$

STT	Tập phổ biến tối đại	Support
1	{66}	4472
2	{40, 42, 49}	7366
3	{33, 40, 49}	5402
4	{39, 40, 49}	6102

Kết quả cho thấy các mã sản phẩm 40 và 49 xuất hiện trong cả ba tập phổ biến tối đại có ba phần tử. Điều này gợi ý rằng 40 và 49 là hai sản phẩm trung tâm trong dữ liệu Retail, thường đóng vai trò cầu nối với các sản phẩm khác như 33, 39 và 42. Riêng sản phẩm 66 tạo thành một tập phổ biến tối đại đơn lẻ, nghĩa là sản phẩm này xuất hiện đủ thường xuyên nhưng không hình thành tổ hợp phổ biến lớn hơn ở ngưỡng 5%.

5.3 Top-10 luật kết hợp theo lift

Từ các tập phổ biến thu được, nhóm sinh luật kết hợp với $minconf = 50\%$ và sắp xếp các luật theo chỉ số lift giảm dần. Top-10 luật kết hợp được trình bày trong Bảng 18. Cột Support là support tuyệt đối của $X \cup Y$.

Bảng 18: Top-10 luật kết hợp trên tập dữ liệu Retail theo lift

STT	Vế trái X	Vế phải Y	Support	Conf.	Lift
1	{42, 49}	{40}	7366	0.82	1.42
2	{40, 42}	{49}	7366	0.65	1.35
3	{33, 40}	{49}	5402	0.64	1.34
4	{39, 49}	{40}	6102	0.77	1.34
5	{42}	{40}	11414	0.76	1.33
6	{42}	{49}	9018	0.60	1.26
7	{39, 40}	{49}	6102	0.59	1.23
8	{49}	{40}	29142	0.69	1.20
9	{40}	{49}	29142	0.57	1.20
10	{33, 49}	{40}	5402	0.67	1.17

Tất cả các luật trong Bảng 18 đều có *confidence* ≥ 0.50 và *lift* > 1 . Vì vậy, các luật này không chỉ thỏa điều kiện lọc ban đầu mà còn thể hiện quan hệ mua kèm tích cực: khi vế trái xuất hiện trong giỏ hàng, khả năng xuất hiện của vế phải tăng lên so với mức thông thường.

5.4 Ý nghĩa kinh doanh của các luật

1. $\{42, 49\} \Rightarrow \{40\}$: Nên gợi ý 40 khi khách đã mua 42 và 49; đây là luật mạnh nhất theo lift.
2. $\{40, 42\} \Rightarrow \{49\}$: Có thể đề xuất 49 như sản phẩm mua kèm cho giỏ hàng chứa 40 và 42.
3. $\{33, 40\} \Rightarrow \{49\}$: Nên dùng 49 làm mặt hàng bán kèm hoặc giảm giá nhẹ cho nhóm khách mua 33 và 40.
4. $\{39, 49\} \Rightarrow \{40\}$: Sản phẩm 40 nên được đề xuất thêm khi giỏ hàng đã có 39 và 49.
5. $\{42\} \Rightarrow \{40\}$: 42 là tín hiệu tốt để gợi ý 40, do confidence đạt 0.76.
6. $\{42\} \Rightarrow \{49\}$: Có thể thử combo 42–49, nhưng mức ưu tiên thấp hơn combo 42–40.
7. $\{39, 40\} \Rightarrow \{49\}$: 49 là lựa chọn phù hợp để bán kèm với cặp 39 và 40.
8. $\{49\} \Rightarrow \{40\}$: Do support rất cao, nên đặt hoặc gợi ý 40 gần 49 để tăng mua kèm.
9. $\{40\} \Rightarrow \{49\}$: Quan hệ 40–49 là hai chiều, phù hợp cho trưng bày gần nhau hoặc combo cơ bản.
10. $\{33, 49\} \Rightarrow \{40\}$: Có thể đề xuất 40 cho khách đã chọn 33 và 49 để tăng giá trị giỏ hàng.

5.5 Nhận xét tổng hợp

Các luật có lift cao nhất đều xoay quanh nhóm sản phẩm 40, 42 và 49. Trong đó, 40 thường là sản phẩm được dự đoán ở vế phải với confidence cao, còn 42 đóng vai trò tín hiệu mạnh để kích hoạt khuyến nghị. Hai sản phẩm 40 và 49 có support rất lớn, vì vậy đây là cặp sản phẩm quan trọng cho các chiến lược trưng bày, bán kèm và quản lý tồn kho.

Về mặt kinh doanh, kết quả có thể được sử dụng theo ba hướng chính. Thứ nhất, xây dựng hệ thống gợi ý sản phẩm theo thời gian thực: khi giỏ hàng chứa 42 và 49, hệ thống ưu tiên đề xuất 40. Thứ hai, thiết kế chương trình khuyến mãi theo combo, đặc biệt cho các bộ 40–42–49, 33–40–49 và 39–40–49. Thứ ba, tối ưu trưng bày và tồn kho bằng cách đặt các sản phẩm có quan hệ mua kèm gần nhau, đồng thời đảm bảo các sản phẩm trung tâm như 40 và 49 luôn có đủ hàng.

Tuy nhiên, các luật kết hợp chỉ phản ánh quan hệ đồng xuất hiện trong dữ liệu, không chứng minh quan hệ nhân quả. Do đó, trước khi áp dụng trực tiếp, doanh nghiệp nên kết hợp thêm thông tin về tên sản phẩm, biên lợi nhuận, mùa vụ, tồn kho và kiểm thử A/B để đánh giá hiệu quả thực tế của các đề xuất bán kèm.

Tài liệu

- [1] R. J. Bayardo, “Efficiently mining long patterns from databases,” in *Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data*, 1998, pp. 85–93. DOI: [10.1145/276304.276316](#)
- [2] D. Burdick, M. Calimlim, and J. Gehrke, “Mafia: A maximal frequent itemset algorithm for transactional databases,” in *Proceedings of the 17th International Conference on Data Engineering (ICDE)*, 2001, pp. 443–452. DOI: [10.1109/ICDE.2001.914857](#)
- [3] K. Gouda and M. J. Zaki, “Genmax: An efficient algorithm for mining maximal frequent itemsets,” *Data Mining and Knowledge Discovery*, vol. 11, no. 3, pp. 223–242, 2005. DOI: [10.1007/s10618-005-0002-x](#)
- [4] M. J. Zaki and K. Gouda, “Fast vertical mining using diffsets,” in *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2003, pp. 326–335. DOI: [10.1145/956750.956788](#)